

Terraform Assignment – AWS Infrastructure Automation

Business Scenario: E-Commerce Application Deployment

Company: ShopEase Technologies Pvt. Ltd.

Problem Statement

ShopEase is planning to deploy its e-commerce application on AWS. The DevOps team wants to use Terraform to automate infrastructure provisioning instead of creating AWS resources manually.

Create a Terraform configuration that provisions the basic AWS infrastructure required for the ShopEase Development Environment.

Infrastructure Requirements

- VPC with CIDR 10.0.0.0/16
- Two public subnets in different Availability Zones
- Internet Gateway
- Public Route Table associated with both public subnets
- Security Group allowing SSH (22), HTTP (80), and HTTPS (443)
- One EC2 instance in a public subnet
- One S3 bucket for application/static files

Terraform Requirements

The solution must use variables instead of hardcoding configuration values.

Create variables for: `aws_region`, `environment`, `project_name`, `vpc_cidr`, `public_subnet_cidrs`, `availability_zones`, `instance_type`, `instance_count`, `allowed_ports`, and `common_tags`.

Use appropriate Terraform concepts such as: AWS Provider, resources and dependencies, input variables, string/number/list/map data types, count or for_each, resource references, outputs, and tags.

Outputs

Create an `outputs.tf` file to display:

- VPC ID
- Subnet IDs
- EC2 Instance ID
- EC2 Public IP
- S3 Bucket Name

Expected Execution

```
terraform init
```

```
terraform validate
```

```
terraform plan
```

```
terraform apply
```

The same Terraform configuration should be designed so that the environment, region, CIDR ranges, EC2 instance type/count, ports, and tags can be changed through variables without modifying the main Terraform resource definitions.

Deliverables

`main.tf`, `variables.tf`, `outputs.tf`, `terraform.tfvars`, and a brief README explaining the Terraform concepts used.